basic optimizations and identify bottlenecks easily. Recent Intel GPA releases now also support Intel Core processors running on Microsoft Windows 7 or 8 target platforms. The Intel GPA System Analyzer now also allows you to target both Android apps and Microsoft Windows applications in the recent releases.

The Intel GPA Monitor launches the application for analysis and other dependent components that are required to run the application, detects the launched application, configures the HUD (Heads Up Display), sets up triggers and application profiles, etc. The Intel GPA System Analyzer GUD allows you to analyze metrics like memory, power, and OpenGL-related metrics, for example, App Resident memory, current charging, Frame Time, etc. It captures frames for further analysis and finds performance bottlenecks. The Intel GPA System Analyzer helps analyze OpenGL ES* applications running on an Intel processor-based Android device that is connected to the host system. The best part about this tool is that it does not require the developer to make any changes to the application code. The wide range of metrics and intuitive user interface allows you to explore and easily determine available functionality. This tool is also well suited for performing experimentation during the development process of the application to identify various approaches for adding and removing visual effects to gauge their performance impact.

SETTING UP THE ADB CONNECTION

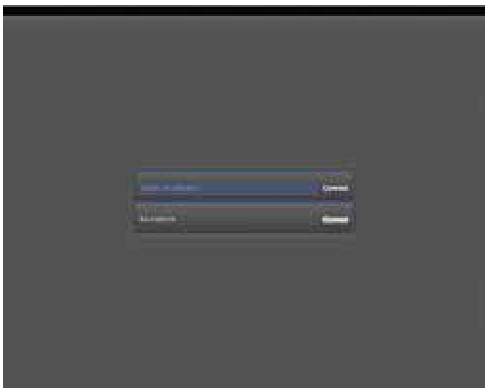
Before setting up the Android Debug Bridge, or ADB, first be sure that the USB Debugging option is enabled in the Settings menu on the Android device. If not done, it can be enabled from the developer options menu in the settings menu of the Android device. Since Beacon Mountain already installs all the required dependencies for you, not much else is required in terms of installing extra software. If the Google Android SDK and the Android USB Driver are not installed, Beacon Mountain will display a message asking you to ensure that these requirements are met before it lets you proceed. To start the connection, plug your device into the host computer with the USB cable and then open the Intel GPA tool's settings dialog by pressing [Ctrl]+[F1] from the Connection window. Here, make sure that you provide the correct path for the ADB tool installed on your system from under the platform tools directory of the Google Android SDK. This is all that Intel GPA requires in order for your device to connect to your Intel processor-based Android device. If you have any problems, you can disconnect the device, run the "adb kill-server" command, and then see the output of ADB devices after re-connecting again. You can also establish a connection over Wi-Fi* to perform analysis wirelessly for Android 4.0 or newer devices.

The Android Application chooser in the GPA tool

PERFORMING THE ANALYSIS

Once connected, the application being analyzed is shown in the list of analyzable applications. The application can be made analyzable by enabling the debugging and Internet connection permission in the Android Manifest.

The main Intel GPA screen is divided into various panels for viewing and controlling parts of the application. The Metrics panel shows you the list of metrics supported by the connected device. This varies depending on the underlying chipset of various devices. When Intel GPA starts, the device connection screen generates the content to add support for JNI functions in the Android application. To run the application that you are developing, you will need to copy the files from the Beacon Mountain installation directory and enter the source where it is relevant. In our example, the native function retrieves the version of the Intel® Integrated Performance Primitives (Intel® IPP) and returns the value to the calling Java function. This is shown in the HelloIPP.c example that follows:



The device connection screen when GPA starts

```
#include <jni.h>
#include "ipp_preview.h"
JNIEXPORT void JNICALL
Java_intel_example_Hel-
loIPP_Hel-   loIPP_IPPVersion(
JNIEnv*  env,  job-  ject  thiz,
jin- tArray *vinfo) {
  // get a pointer to the Java
array      jint *tmp  =  (*env)-
>GetIntArrayElements(env, vinfo, 0);
    // call IPP's utility function
to get library version info
  const    IppLibraryVersion
*lib=ippGetLibVersion();   // copy
this info into the array (visible
from Java caller)
       tmp[0]=lib->major;
       tmp[1]=lib->minor;
    // release the pointer   (*env)-
>ReleaseIntArrayElements(env, vinfo,
tmp, 0);
  }
```

The calling function can be called in Java as follows:

```
  package intel.example.HelloIPP;
  import android.app.Activity;
  import android.widget.TextView;
  import android.os.Bundle; public
class HelloIPP extends Activity
  public class HelloIPP extends
  Activity
  {
   public native void
  IPPVersion(int[] vinfo);
   @Override
   public void onCreate(Bundle save-
dInstanceState)
    {
                  super.
onCreate(savedInstanceState);
   // Call the native function which
queries the IPP version info
    int[] ippversion=new int[2];
    IPPVersion(ippversion);
```